

Pipelining in RISC Processors

1. Introduction to Pipelining

Pipelining is a processor design technique where multiple instructions are overlapped during execution. Instead of executing one instruction at a time, the processor divides execution into stages and processes several instructions simultaneously. This significantly improves throughput while keeping clock periods short.

Key Points:

- Improves instruction throughput
- Does NOT reduce the latency of a single instruction
- Ideal CPI approaches 1 after pipeline fill
- Comparable to an industrial assembly line

2. Why RISC Architectures Suit Pipelining

RISC processors are designed with features that naturally support pipelining:

- Fixed-length instructions simplify decoding
- Load/store architecture reduces memory conflicts
- Few addressing modes minimize control complexity
- Hardwired control enables faster execution

3. Classic 5-Stage RISC Pipeline

Instruction Fetch (IF): Instruction is read from memory using the Program Counter.

Instruction Decode (ID): Opcode is interpreted, registers are read, and control signals are generated.

Execute (EX): ALU performs arithmetic/logic operations and calculates branch targets.

Memory Access (MEM): Data memory is accessed for load/store instructions.

Write Back (WB): Results are written into the register file.

4. Pipeline Performance

Without Pipeline: Execution Time = $n \times k \times T$

With Pipeline: Execution Time = $(n + k - 1) \times T$

Where n = number of instructions, k = number of stages, and T = clock cycle time.

Ideal Speedup: k (rare in practice due to hazards).

5. Pipeline Hazards

Structural Hazards: Occur when hardware resources are insufficient. Solution: separate instruction and data caches.

Data Hazards: Occur when instructions depend on previous results. RAW is the most common type. Solved using forwarding/bypassing or pipeline stalls.

Control Hazards: Caused by branch instructions. Solutions include branch prediction, delayed branching, and pipeline flushing.

6. Pipeline Stall (Bubble)

A stall inserts an empty clock cycle to resolve hazards. Excessive stalls significantly degrade performance.

7. Throughput vs Latency

Latency: Time taken for one instruction to complete.

Throughput: Number of instructions completed per unit time.
Pipelining improves throughput, not latency.

8. Advanced Concepts

Superpipelining: Uses more stages to increase clock frequency but raises branch penalties.

Superscalar Architecture: Issues multiple instructions per clock cycle using out-of-order execution and register renaming.

Pipeline Depth Trade-off: Deeper pipelines improve speed but increase misprediction penalties.

9. Numerical Example

A processor executes instructions in 5 cycles of 10 ns each. For 100 instructions:

Without pipeline = $100 \times 5 \times 10 = 5000$ ns

With pipeline = $(100 + 5 - 1) \times 10 = 1040$ ns

Speedup ≈ 4.8

10. Expected University Questions

- Explain pipelining with a diagram.
- Derive pipeline speedup.
- Discuss types of hazards and mitigation techniques.
- Compare throughput and latency.
- Explain why RISC architectures favor pipelining.